

LAPORAN PRAKTIKUM
PEMROGRAMAN BERBASIS OBJEK
Ekspansi Produk Keuangan (Inheritance)

Disusun Oleh:

Nofri Ilham

2511531013

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Ezza Andini



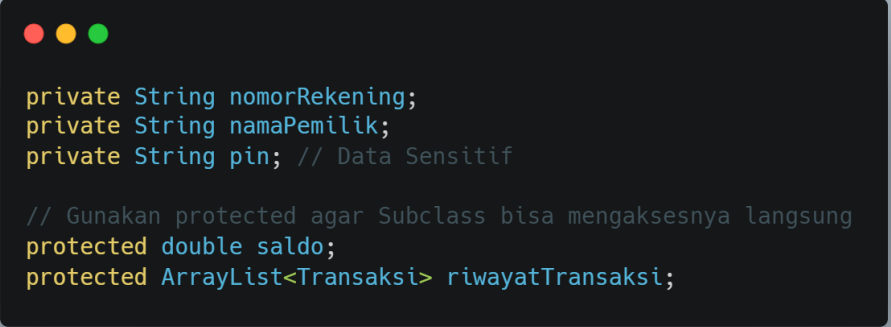
DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

A. Tahapan Pelaksanaan Praktikum

1. Memastikan Akses Modifier di Rekening.java

Sebelum membuat produk turunan, class Rekening perlu direfaktori menjadi superclass agar bisa diwarisi. Atribut nomorRekening dan pin tetap dibiarkan private, karena keduanya adalah data identitas dan data rahasia yang tidak perlu dan tidak boleh diakses langsung meskipun oleh subclass. Sebaliknya, atribut saldo dan riwayatTransaksi diubah aksesnya menjadi protected.



```
private String nomorRekening;  
private String namaPemilik;  
private String pin; // Data Sensitif  
  
// Gunakan protected agar Subclass bisa mengaksesnya langsung  
protected double saldo;  
protected ArrayList<Transaksi> riwayatTransaksi;
```

Perubahan ke protected ini diperlukan karena subclass seperti RekeningTabungan dan RekeningGiro nantinya perlu memanipulasi saldo (misalnya menambah bunga) dan mencatat riwayat transaksi secara langsung, tanpa harus membuka akses tersebut ke publik.

2. Membuat Subclass RekeningTabungan

Class RekeningTabungan dibuat dengan kata kunci extends Rekening, artinya RekeningTabungan mewarisi seluruh atribut protected/public dan method milik Rekening. Relasi ini disebut relasi is-a: Tabungan is-a Rekening.

```

public class RekeningTabungan extends Rekening {

    // Atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    // Constructor Subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String
pinAwal, double sukuBunga) {
        // super() memanggil constructor kelas induk (Rekening). WAJIB berada di
baris pertama
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * sukuBunga / 100;
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" +
nominalBunga);
    }
}

```

Atribut sukuBunga bersifat spesifik dan hanya dimiliki oleh Tabungan, sehingga dideklarasikan private di class ini, bukan di Rekening. Pemanggilan super(nomor, nama, saldoAwal, pinAwal) wajib berada di baris pertama constructor, karena inilah cara Java menjalankan proses inisialisasi milik Superclass (termasuk validasi PIN dan inisialisasi riwayatTransaksi) sebelum baris kode di subclass dieksekusi.

Method tambahBungaAkhirBulan() dapat langsung mengakses saldo dan riwayatTransaksi tanpa Getter atau Setter, karena kedua atribut tersebut sudah diubah menjadi protected pada Langkah 1. Bunga dihitung dari saldo dikali persentase suku bunga, hasilnya ditambahkan ke saldo, lalu dicatat sebagai transaksi baru berjenis "Bunga" dengan prefiks ID TRX-B-.

3. Membuat Subclass RekeningGiro

Class RekeningGiro juga dibuat dengan extends Rekening, untuk merepresentasikan rekening bisnis yang memiliki fasilitas Batas Overdraft (limit pinjaman).

```

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal,
double batasOverdraft) {
        // Memanggil inialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}

```

RekeningGiro memanggil `super(...)` pada baris pertama constructor untuk menjalankan inialisasi dasar milik Rekening. Atribut `batasOverdraft` disimpan sebagai `private` karena hanya relevan untuk produk Giro, dan disediakan Getter `getBatasOverdraft()` agar nilainya bisa dibaca dari luar tanpa bisa diubah sembarangan.

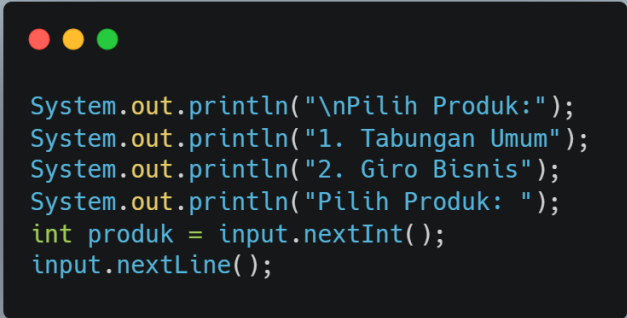
B. Latihan/Tugas

1. Pembuatan Multi-Produk

Modifikasi case 1: Buka Rekening Baru di `Main.java`.

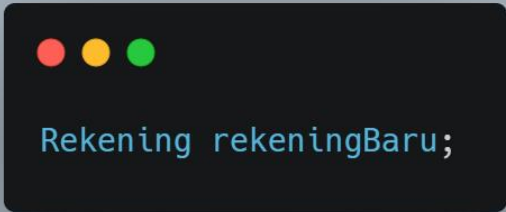
1. Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1.Tabungan Umum | 2. Giro Bisnis"
2. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek `RekeningTabungan`.
3. Jika user memilih Giro, minta input `batasOverdraft` (limit pinjaman), lalu instansiasi objek `RekeningGiro`.
4. Simpan objek tersebut ke dalam `daftarRekening` (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

Pertama, dilakukan pengembangan pada Menu 1 (Buka Rekening Baru) di `Main.java`. Pengembangan ini bertujuan agar sistem mendukung lebih dari satu jenis produk rekening, yaitu Tabungan Umum dan Giro Bisnis, sehingga objek yang dibuat sesuai dengan pilihan user.



```
System.out.println("\nPilih Produk:");  
System.out.println("1. Tabungan Umum");  
System.out.println("2. Giro Bisnis");  
System.out.println("Pilih Produk: ");  
int produk = input.nextInt();  
input.nextLine();
```

Variabel rekeningBaru dideklarasikan bertipe Rekening (Superclass), bukan tipe subclass, agar dapat menampung objek RekeningTabungan maupun RekeningGiro sesuai pilihan user. Ini adalah penerapan upcasting, karena referensi bertipe superclass digunakan untuk menyimpan objek bertipe subclass.



```
Rekening rekeningBaru;
```

Apabila user memilih produk 1 (Tabungan), program meminta input suku bunga dalam persen, lalu membuat objek RekeningTabungan menggunakan constructor yang menerima parameter tambahan sukuBunga.

```

if (produk == 1) {
    System.out.println("Masukkan Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
}

```

Apabila user memilih produk 2 (Giro), program meminta input batas overdraft, lalu membuat objek RekeningGiro menggunakan constructor yang menerima parameter tambahan batasOverdraft.

```

else if (produk == 2) {
    System.out.println("Masukkan Batas Overdraft: ");
    double batasOverdraft = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
}

```

Jika user memasukkan pilihan produk selain 1 atau 2, program menampilkan pesan "Pilihan produk tidak valid!" dan proses pembuatan rekening dibatalkan dengan break.

```

else {
    System.out.println("Pilihan produk tidak valid!");
    break;
}

```

Setelah objek berhasil dibuat, baik RekeningTabungan maupun RekeningGiro dimasukkan ke dalam daftarRekening yang bertipe ArrayList<Rekening>, dan dijadikan akunAktif.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains two lines of Java code:

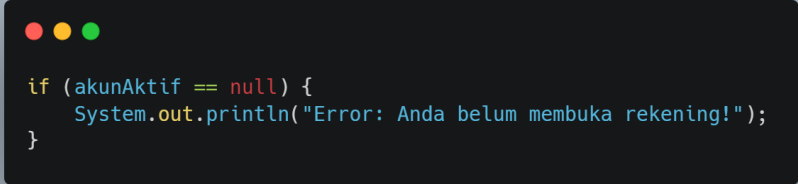
```
daftarRekening.add(rekeningBaru);  
akunAktif = rekeningBaru;
```

```
daftarRekening.add(rekeningBaru);  
akunAktif = rekeningBaru;
```

2. Simulasi Waktu Akhir Bulan

1. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
2. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
3. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
4. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

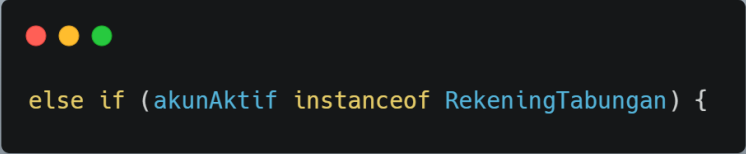
Pada tugas kedua, dilakukan penambahan Menu 8 (Simulasi Akhir Bulan) di Main.java. Pada program, terlebih dahulu dilakukan pengecekan apakah akunAktif masih null, untuk mencegah error jika belum ada rekening yang dibuka.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains a Java code snippet for a null check:

```
if (akunAktif == null) {  
    System.out.println("Error: Anda belum membuka rekening!");  
}
```

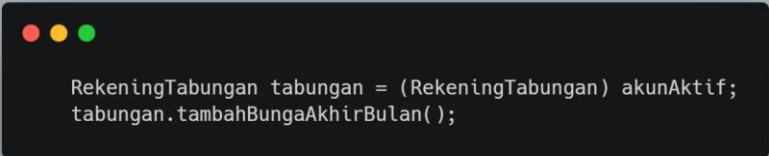
```
if (akunAktif == null) {  
    System.out.println("Error: Anda belum membuka rekening!");  
}
```

Jika rekening sudah ada, program memeriksa jenis objek sebenarnya dari akunAktif menggunakan operator instanceof. Ini agar dapat diketahui akunAktif bertipe rekening (RekeningTabungan) bukan (RekeningGiro)



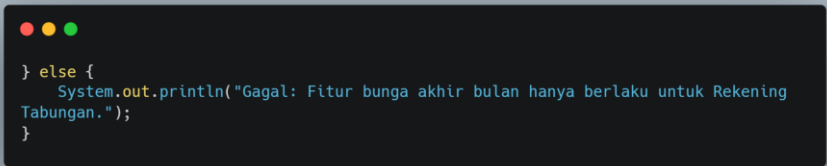
```
else if (akunAktif instanceof RekeningTabungan) {
```

Apabila hasil pengecekan bernilai true, artinya objek aslinya memang RekeningTabungan, maka dilakukan downcasting secara eksplisit, yaitu mengubah referensi bertipe Rekening kembali menjadi RekeningTabungan agar method tambahBungaAkhirBulan() (yang hanya ada di subclass ini) dapat dipanggil.



```
RekeningTabungan tabungan = (RekeningTabungan) akunAktif;  
tabungan.tambahBungaAkhirBulan();
```

Apabila akunAktif bukan merupakan RekeningTabungan (misalnya objeknya adalah RekeningGiro), maka downcasting tidak dilakukan sama sekali, dan program menolak permintaan tersebut.



```
} else {  
    System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening  
    Tabungan.");  
}
```

C. Link Repository

https://github.com/nofriilham/PrakPBO_C_2511531013/tree/Pertemuan4